CAMPUS CONNECT

A SOCIAL NETWORKING MOBILE APPLICATION FOR CAMPUS COMMUNITIES

Prepared in the partial fulfillment of the Summer Internship Program

AT

Under the guidance of

Mrs. Sumana Bethala, APSSDC

Mr. J. Lovababu, APSSDC

Submitted by

Duddugunta Moulendra Reddy, 24KB5A0508, N.B.K.R Institute of Science and Technology

Nelluru Jaswanth, 23KB1A05E2, N.B.K.R Institute of Science and Technology

Sathi Devi Vara Prasad Reddy, 24KB1A05GP, N.B.K.R Institute of Science and Technology

Seelam Dudhumdeswari, Y23ACS525, Bapatla Engineering College

Marrapu Sravya, Y23CS2663, Bapatla Women's Engineering College

Gurijala Avinash, Y23AIT438, Bapatla Engineering College

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who have contributed to the successful completion of our project, Campus Connect, developed as part of the Summer Internship Program. This opportunity has been an enriching and transformative experience for us, and we are truly thankful for the support, guidance, and encouragement we have received along the way.

We extend our sincere regards to Mrs. Sumana Bethala and Mr. J. Lovababu, our mentors, for providing us with valuable insights, constant guidance, and unwavering support throughout the duration of the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

We would also like to thank our respective institutions — N.B.K.R Institute of Science and Technology, Bapatla Engineering College, and Bapatla Women's Engineering College — for fostering an environment of learning and collaboration that helped us bring this project to life.

In conclusion, we are honoured to have been a part of this internship program, and we look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Thank you.

Sincerely,

Duddugunta Moulendra Reddy, 24KB5A0508, moulendramoulireddy@gmail.com
Nelluru Jaswanth, 23KB1A05E2, nellurujaswanth2004@gmail.com
Sathi Devi Vara Prasad Reddy, 24KB1A05GP, sathidevivaraprasad53383@gmail.com
Seelam Dudhumdeswari, Y23ACS525, dudhum2005@gmail.com
Marrapu Sravya, Y23CS2663, sravyamarappu@gmail.com
Gurijala Avinash, Y23AIT438, gurijalaabhi555@gmail.com

ABSTRACT

This project presents the design and development of Campus Connect, a complete, production-ready social networking mobile application built for campus communities using React Native (Expo) and Firebase. The primary goal of the application is to bring students, faculty, and campus organizers onto a single real-time platform where they can share updates, discover events, and communicate directly, replacing the fragmented mix of messaging apps, noticeboards, and social media groups typically used for campus communication.

The application is built on a cross-platform mobile framework (React Native with Expo) and uses file-based routing through Expo Router to organize authentication, feed, events, chat, notifications, and profile screens. Firebase serves as the complete backend: Firebase Authentication manages secure sign-up and login, Cloud Firestore acts as the real-time NoSQL database for posts, events, chats, and notifications, Firebase Storage handles image and file uploads, and Firebase Cloud Messaging (FCM) together with Expo Notifications delivers push alerts for likes, comments, messages, and events.

Core features include a social feed with posts, likes, and comments; an events module supporting browsing, filtering, and registration; real-time one-to-one and group chat with file sharing; a unified notification system; and a searchable, theme-aware profile experience with full dark mode support. State is managed on the client using Zustand, while Firestore Security Rules and role-based access (an isAdmin flag) enforce controlled access to sensitive operations such as event creation.

By combining a modern mobile front end with a serverless, real-time backend, Campus Connect demonstrates how cloud-backed mobile architectures can deliver a scalable, low-maintenance, and highly interactive community platform. In addition to its Firebase-based core, the platform integrates a hybrid AWS layer for analytics, content moderation, and notification orchestration: Amazon S3 stores detailed JSON activity logs as a queryable data lake, Lambda functions exposed through API Gateway host REST endpoints for moderation, image analysis, notifications, and analytics, Amazon Rekognition screens uploaded media for unsafe content and extracts text and tags, Amazon Comprehend analyzes post text for sentiment, key phrases, language, and toxicity, and Amazon Pinpoint together with SNS orchestrates targeted push notifications and campus-wide

broadcasts. This hybrid design provides a strong foundation for future enhancements such as richer analytics dashboards, automated moderation, and AI-assisted content recommendations.

TABLE OF CONTENTS

1. INTRODUCTION

In today's digital era, campus life spans a scattered mix of WhatsApp groups, printed noticeboards, email chains, and third-party social media pages, none of which are purpose-built for the needs of a college community. This project, titled Campus Connect, addresses that gap by delivering a single, purpose-built mobile application where students and staff can post updates, discover and register for events, chat in real time, and receive timely notifications, all within one cohesive experience.

Modern student communities expect the same real-time, always-available experience they get from mainstream social apps: instant feeds, live chat, push alerts, and a polished, responsive interface that works equally well on Android and iOS. Campus Connect is designed around this expectation. It is built using React Native with Expo, allowing a single TypeScript codebase to run natively on both major mobile platforms, and it is powered entirely by Firebase, giving the application a serverless, auto-scaling backend without the need to provision or maintain any servers.

The system allows any registered user to create posts with text, images, or file attachments, like and comment on posts from others, and search across posts, events, and users. Campus event organizers can publish events with details such as date, location, and category, and students can browse, filter, and register for events they are interested in. Real-time chat, available as both direct messages and group conversations, lets users coordinate instantly, with support for file sharing within conversations.

A key design decision was to keep the architecture fully serverless on the backend side: Firebase Authentication handles identity, Cloud Firestore provides a real-time NoSQL data layer, Firebase Storage manages media and file uploads, and Firebase Cloud Messaging delivers push notifications. Firestore Security Rules and a simple role flag on each user document (isAdmin) enforce access control, for example restricting event creation to administrators.

This report documents the methodology, technology stack, system architecture, implementation details, and results of building Campus Connect, and closes with a discussion of the benefits realized and directions for future enhancement, such as an admin analytics dashboard, richer content moderation, and AI-driven recommendations.

2. METHODOLOGY

The development of Campus Connect followed a structured, iterative methodology to ensure the project met its functional and technical goals. The process was organized into requirements gathering, system design, implementation, testing, deployment, and refinement, mirroring standard practice for cloud-backed mobile application development. Each phase is described below.

2.1. Requirements Gathering

The project began with an analysis of the everyday communication problems faced by campus communities: scattered event information, no unified way to share updates, and no dedicated real-time chat tied to campus identity. These discussions shaped the core requirements: a unified feed, an events module with registration, real-time chat, push notifications, and a smooth, theme-aware user experience with dark mode support.

2.2. Design

Based on the requirements, a mobile-first, real-time architecture was designed using React Native (Expo) for the client and Firebase for the backend. Expo Router was chosen for file-based navigation, organizing the app into authentication, tab-based, and detail-view route groups. Firestore's document/collection model was used to design schemas for users, posts, events, chat rooms, and notifications, with sub-collections for comments and messages to keep read and write patterns efficient.

2.3. Implementation

Implementation translated the design into working code. Authentication screens (login, registration, forgot password) were built against Firebase Auth. The feed, events, chat, notifications, and profile tabs were implemented as separate route groups, each backed by a dedicated Zustand store and service module that encapsulates Firestore and Storage calls. Real-time listeners on Firestore collections keep the feed and chat views synchronized across devices without manual polling.

2.4. Database Management

A NoSQL approach was adopted using Cloud Firestore for flexible, horizontally scalable data storage. Collections were designed for users, posts (with a comments sub-collection), events, chatRooms (with a messages sub-collection), and notifications. This schema supports fast reads for feed rendering and chat display while keeping write operations isolated to the relevant sub-collection, preserving data integrity and consistency.

2.5. Testing

Testing covered functional flows such as registration and login, post creation with attachments, event registration, and real-time message delivery between multiple devices. Particular attention was paid to Firestore Security Rules, verifying that users could only modify their own posts and profile data, and that only admin-flagged accounts could create events.

2.6. Deployment

The application was configured for deployment using Expo Application Services (EAS), with environment variables managing Firebase configuration across builds. Firestore Security Rules were published directly through the Firebase Console, and push notification credentials were configured for both Expo Notifications and Firebase Cloud Messaging to support Android and iOS delivery.

2.7. Iterative Refinement

Throughout development, feedback loops refined the notification formatting, dark mode theming, and chat performance. The architecture was reviewed periodically to keep the codebase modular — separating stores, services, hooks, and components — so the application remains maintainable and ready for future features such as an admin dashboard or content moderation tools.

The methodology adopted ensured a real-time, scalable, and maintainable mobile application, successfully demonstrating how a serverless cloud backend can support a full-featured campus social platform.

3. TECHNOLOGY STACK

Campus Connect was built using a modern, cross-platform mobile stack paired with a fully managed, serverless backend. The following technologies and services were used throughout the implementation:

3.1. Mobile Framework

- React Native with Expo (SDK 50+): Enables a single TypeScript codebase to run natively on both Android and iOS.
- Expo Router: File-based routing used to organize authentication, tab, and detail-view screens.

3.2. Backend & Data Services (Firebase)

- Firebase Authentication: Manages email/password registration, login, and password recovery.
- Cloud Firestore: Real-time NoSQL database storing users, posts, events, chat rooms, and notifications.
- Firebase Storage: Stores uploaded images and file attachments for posts and chat messages.
- Firebase Cloud Messaging (FCM): Delivers push notifications in conjunction with Expo Notifications.

3.3. State Management & Styling

- Zustand: Lightweight state management for auth, posts, events, chat, and notifications.
- NativeWind: Tailwind CSS utility styling adapted for React Native, used across all screens including dark mode.

3.4. Supporting Libraries

- Expo Vector Icons (Ionicons): Icon set used throughout the interface.
- Expo Image Picker & Expo Document Picker: Enable image and file attachment selection from the device.

3.5. Programming Languages

- TypeScript: Primary language for application logic and components.
- JavaScript: Used for configuration and tooling scripts.

3.6. Security & Access

Firestore Security Rules and a role-based isAdmin flag on the user document enforce least-privilege access, ensuring users can only modify their own data and that only administrators can create events.

3.7. Cloud Analytics, Moderation & Notification Layer (AWS)

Alongside its Firebase core, Campus Connect integrates a hybrid AWS layer that handles activity analytics, content moderation, and notification orchestration:

- Amazon S3: Acts as a data lake storing detailed JSON logs of student activity events (views, likes, posts, signups) for downstream query tools such as Athena and QuickSight.
- AWS Lambda + Amazon API Gateway: Provides the serverless REST API backend hosting key endpoints for content moderation, image analysis, notifications, and analytics.
- Amazon Rekognition: Analyzes uploaded media for safety, flagging nudity or violence, performs text extraction (OCR), and generates descriptive tags.
- Amazon Comprehend: Examines post text to identify sentiment, detect key phrases, classify language, and filter toxic or abusive content.
- Amazon Pinpoint / Amazon SNS: Orchestrates targeted push notifications and campus-wide announcement broadcasts by mapping user registration tokens to notification topics.

4. SYSTEM DESIGN / ARCHITECTURE

Campus Connect follows a hybrid, cloud-native architecture. The core application — authentication, feed, events, and chat — runs on a real-time client–backend-as-a-service model powered by Firebase, while a parallel AWS layer handles activity analytics, content moderation, and notification orchestration. The two layers communicate through REST endpoints: the mobile client and Firebase Cloud Functions forward relevant events (new posts, uploaded media, activity logs) to the AWS-hosted API for analysis, moderation, and logging.

This section outlines the core components of both layers and their interactions in the architecture.

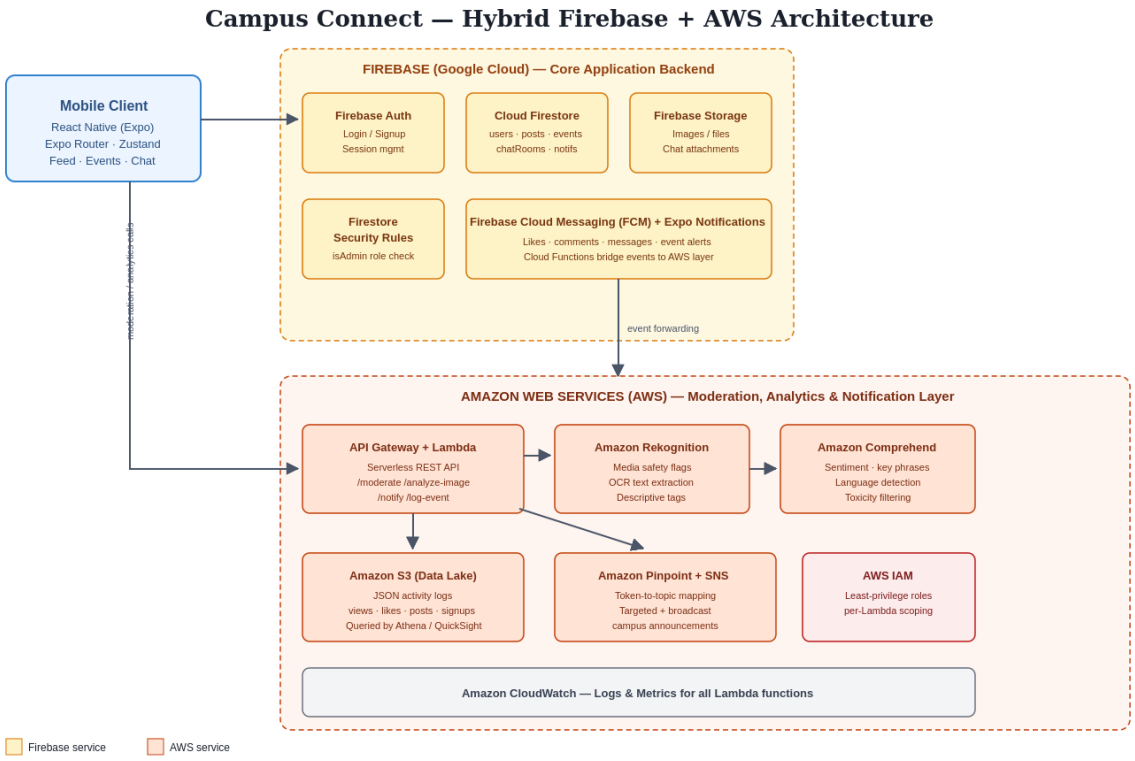

Figure 1. Overall architecture — mobile client, Firebase core, and the AWS moderation/analytics/notification layer

4.1. Client Layer – React Native (Expo)

The mobile client is built with React Native and Expo, using Expo Router for file-based navigation. Screens are organized into route groups: (auth) for login, registration, and password recovery; (tabs) for the feed, events, chat, notifications, and profile; and dynamic routes for individual chat rooms, posts, and events. Components such as PostCard, EventCard, ChatBubble, and NotificationItem render the corresponding data from Firestore.

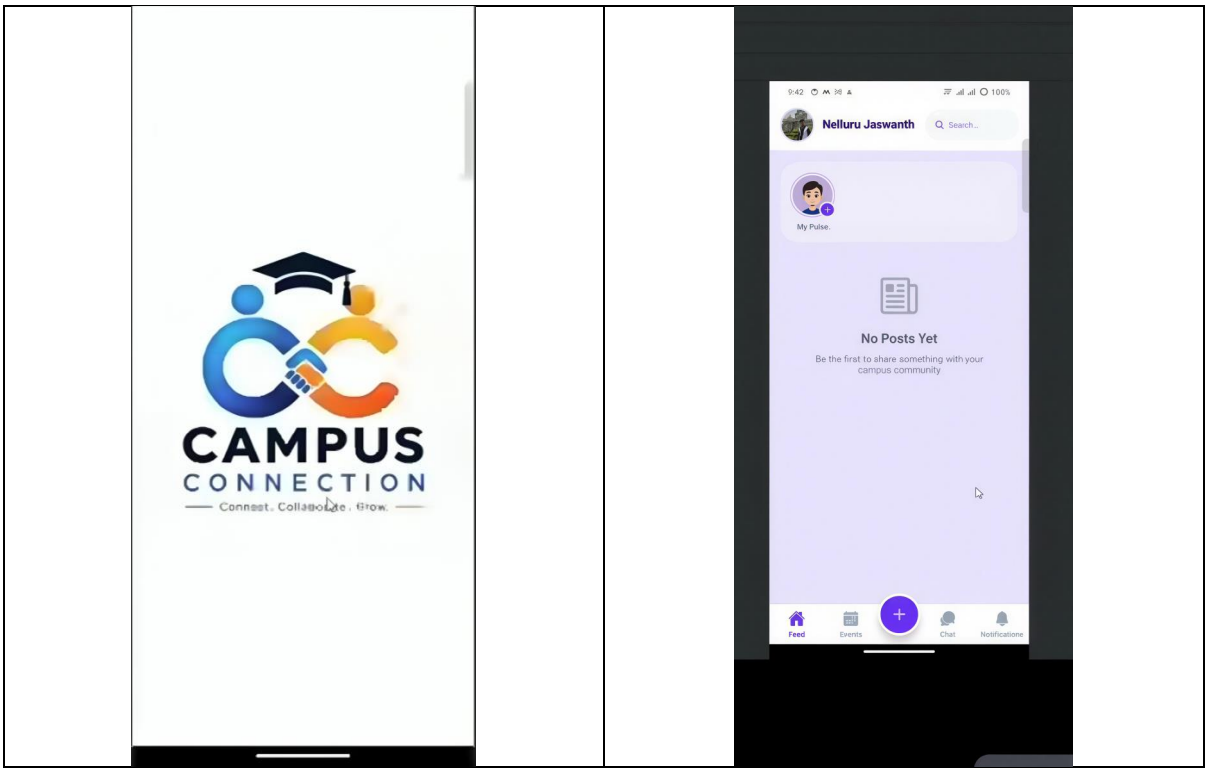

Figure 2. App launch screen and bottom tab navigation across feed, events, chat, notifications, and profile

4.2. Authentication Layer – Firebase Authentication

Firebase Authentication manages user identity using email and password credentials. On successful registration, a corresponding user document is created in Firestore capturing profile details such as name, department, year, avatar, and the isAdmin role flag. The authStore and useAuth hook keep the client's authentication state in sync with Firebase Auth's session state.

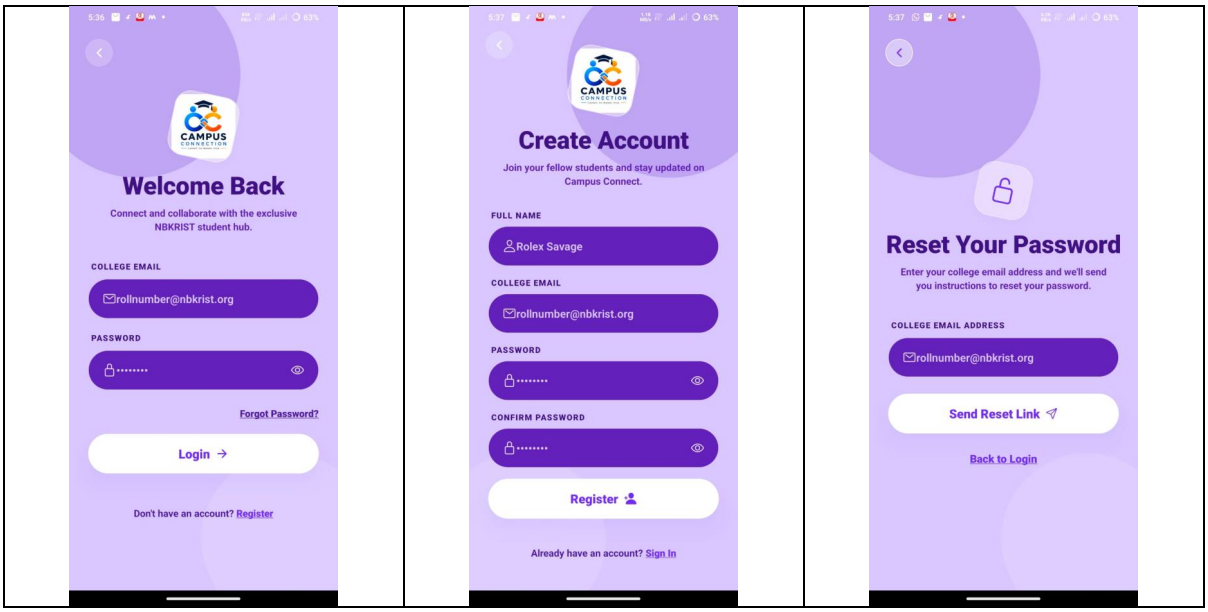

Figure 3. Login, registration, and forgot-password screens

4.3. Real-Time Data Layer – Cloud Firestore

Cloud Firestore is the primary database, structured into five main collections: users, posts (with a comments sub-collection), events, chatRooms (with a messages sub-collection), and notifications. Firestore's real-time listeners allow the feed, chat, and notification screens to update instantly across all connected devices whenever the underlying data changes, without any manual refresh or polling.

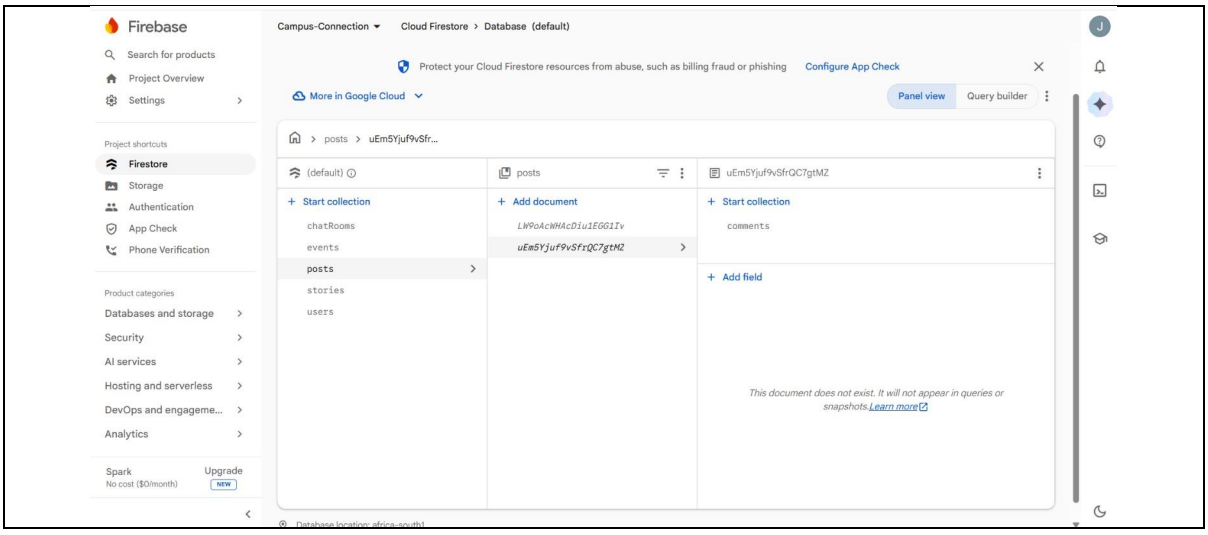

Figure 4a. Firestore console — posts collection with comments sub-collection

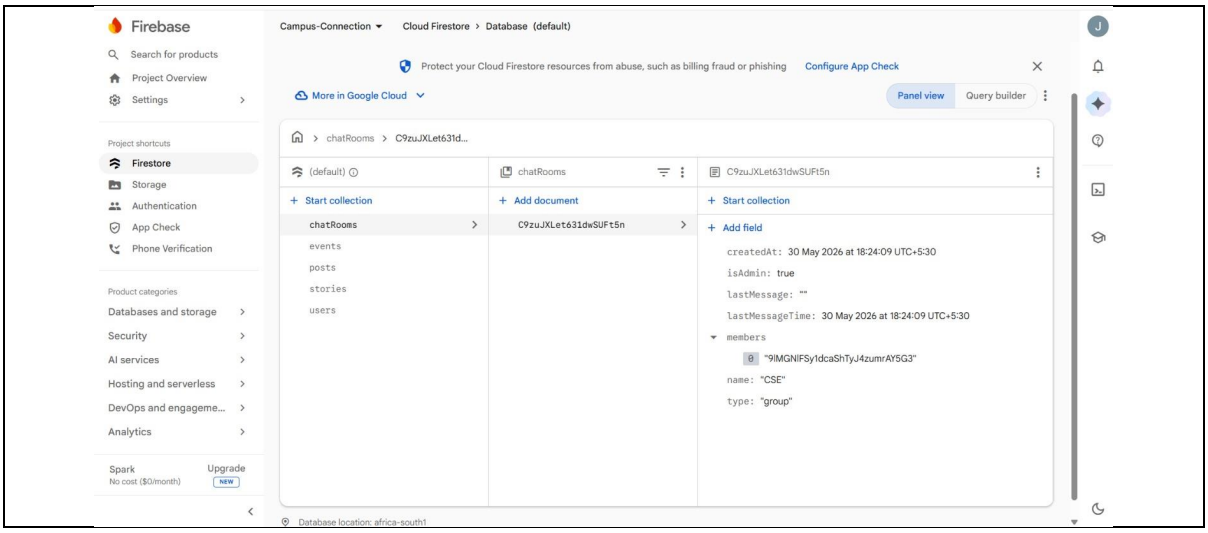

Figure 4b. Firestore console — chatRooms collection document

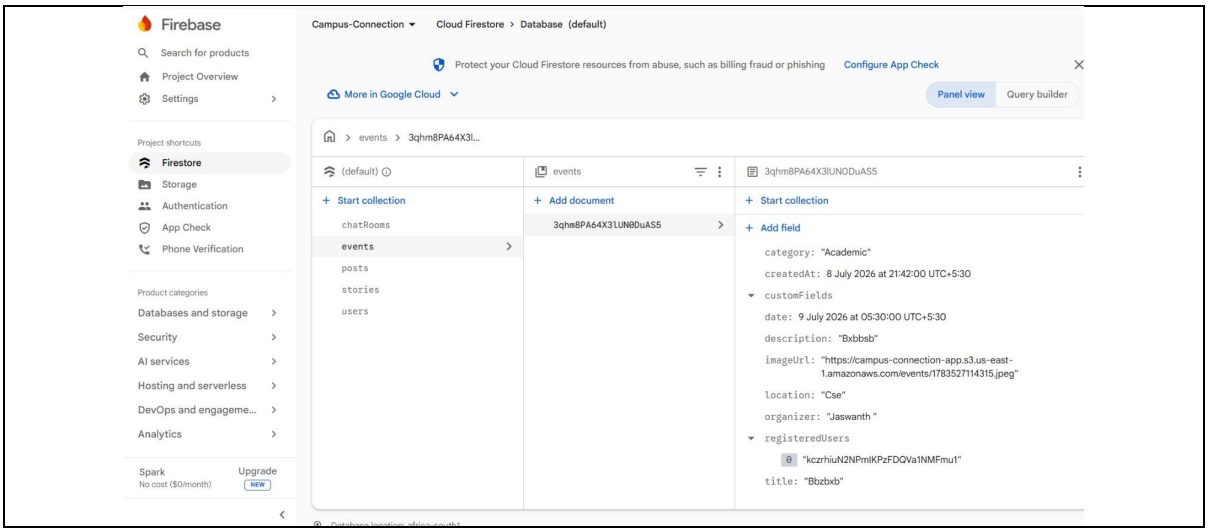

Figure 4c. Firestore console — events collection document

4.4. Storage Layer – Firebase Storage

Firebase Storage handles all binary content, including post images, post file attachments, and files shared within chat messages. Uploaded files are referenced in Firestore documents via secure download URLs, keeping the database layer lightweight while Storage manages the actual file bytes.

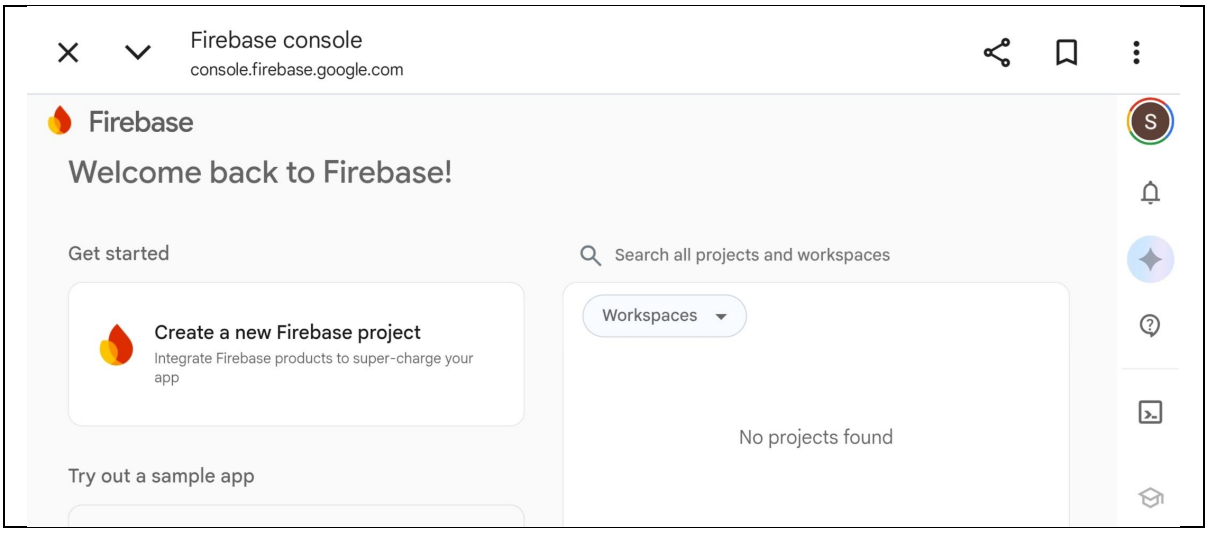

Figure 5. Firebase console — Campus-Connection project workspace

4.5. Notification Layer – FCM and Expo Notifications

When a post is liked or commented on, a chat message is received, or an event is published, the corresponding service writes a document to the notifications collection and triggers a push notification through Firebase Cloud Messaging in combination with Expo Notifications, ensuring users are alerted in real time even when the app is in the background.

4.6. Security and Access Control – Firestore Security Rules

Firestore Security Rules enforce least-privilege access at the database level: users can read public content but can only write to their own posts, profile, and messages within chat rooms they belong to. The isAdmin flag on the user document is checked within these rules to restrict event creation to authorized accounts.

4.7. State Management – Zustand

On the client, Zustand stores (authStore, postStore, eventStore, chatStore, notificationStore) hold the application's in-memory state, subscribing to Firestore listeners through the corresponding service modules and exposing simple actions to components, keeping UI code decoupled from backend logic.

4.8. Hybrid Analytics, Moderation & Notification Layer – AWS

Running alongside the Firebase core, a dedicated AWS layer handles three responsibilities that benefit from AWS's managed AI and analytics services: activity logging, content moderation, and large-scale notification delivery.

Whenever a student views, likes, posts, or signs up, the corresponding event is forwarded as a structured JSON record and written to an Amazon S3 bucket acting as a data lake. This raw event log is queryable through Amazon Athena and visualized through Amazon QuickSight, enabling usage analytics — such as most-engaged content or peak activity times — without impacting the primary Firestore database.

All AWS-side functionality is exposed through a serverless REST API built with AWS Lambda functions fronted by Amazon API Gateway. Dedicated endpoints handle content moderation, image analysis, notification dispatch, and analytics queries, keeping this logic decoupled from the Firebase-hosted mobile backend.

When a post includes an image or video, the media is passed to Amazon Rekognition, which flags unsafe content such as nudity or violence, extracts embedded text through OCR, and generates descriptive tags that can be used for search and categorization. In parallel, the post's text content is passed to Amazon Comprehend, which detects sentiment, extracts key phrases, classifies the language, and flags toxic or abusive language before the post is allowed to go live.

Push notifications and campus-wide announcements are orchestrated through Amazon Pinpoint together with Amazon SNS. User device registration tokens are mapped to notification topics, allowing the system to send both individually targeted alerts (likes, comments, messages) and broadcast announcements to the entire campus community at scale.

Together, this hybrid design keeps the Firebase layer focused on real-time, low-latency application data, while the AWS layer takes on data-intensive analytics and AI-driven moderation — tasks better suited to AWS's managed machine learning and big-data services.

5. IMPLEMENTATION

The implementation of Campus Connect translates the architectural design into a fully functional cross-platform mobile application. This section outlines how each layer was configured and developed to work together in a real-time, event-driven workflow.

5.1. Project Structure

The codebase is organized for clarity and separation of concerns: the app/ directory holds Expo Router routes grouped into (auth) and (tabs), along with dynamic routes for chat rooms, posts, and events; components/ holds reusable UI elements; store/ holds Zustand state slices; services/ wraps all Firebase interactions; hooks/ exposes reusable logic such as useAuth, usePosts, useChat, and useTheme; and constants/ and types/ hold shared configuration and TypeScript types.

5.2. Authentication Flow

Registration and login screens call authService, which wraps Firebase Authentication methods. On successful sign-up, a new document is created in the users collection with fields such as uid, name, email, avatar, department, year, bio, fcmToken, darkMode, and isAdmin. The forgot-password screen triggers Firebase's password-reset email flow.

5.3. Feed, Posts, and Comments

The feed screen subscribes to the posts collection ordered by creation time. Creating a post uploads any attached image or file to Firebase Storage via storageService, then writes a document to posts containing the author's details, content, optional imageUrl and fileUrl, an empty likes array, and a tags array. Before a post is published, its text and media are sent to the AWS moderation API: Amazon Rekognition screens images for unsafe content and extracts tags, while Amazon Comprehend checks the text for toxicity, sentiment, and language. Posts that fail moderation are withheld from the feed. Liking a post updates the likes array; commenting writes a new document into the post's comments sub-collection and increments commentsCount.

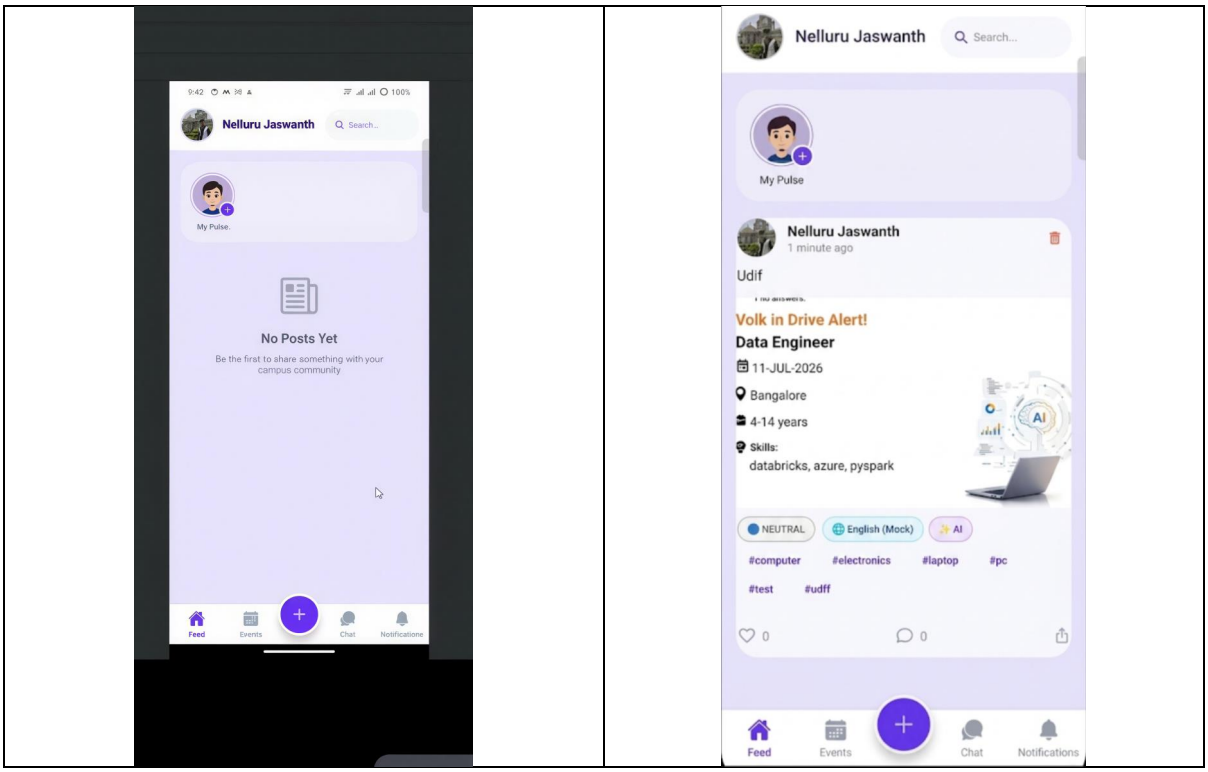

Figure 6. Feed screen — empty state, and a live post showing AI-generated hashtags, sentiment label, and like/comment/share actions

5.4. Events Module

The events screen lists documents from the events collection, supporting filtering by category (Academic, Cultural, Sports, Workshop, Other). Registering for an event adds the user's id to the event's registeredUsers array. Event creation is gated behind the isAdmin flag, both in the UI and in Firestore Security Rules.

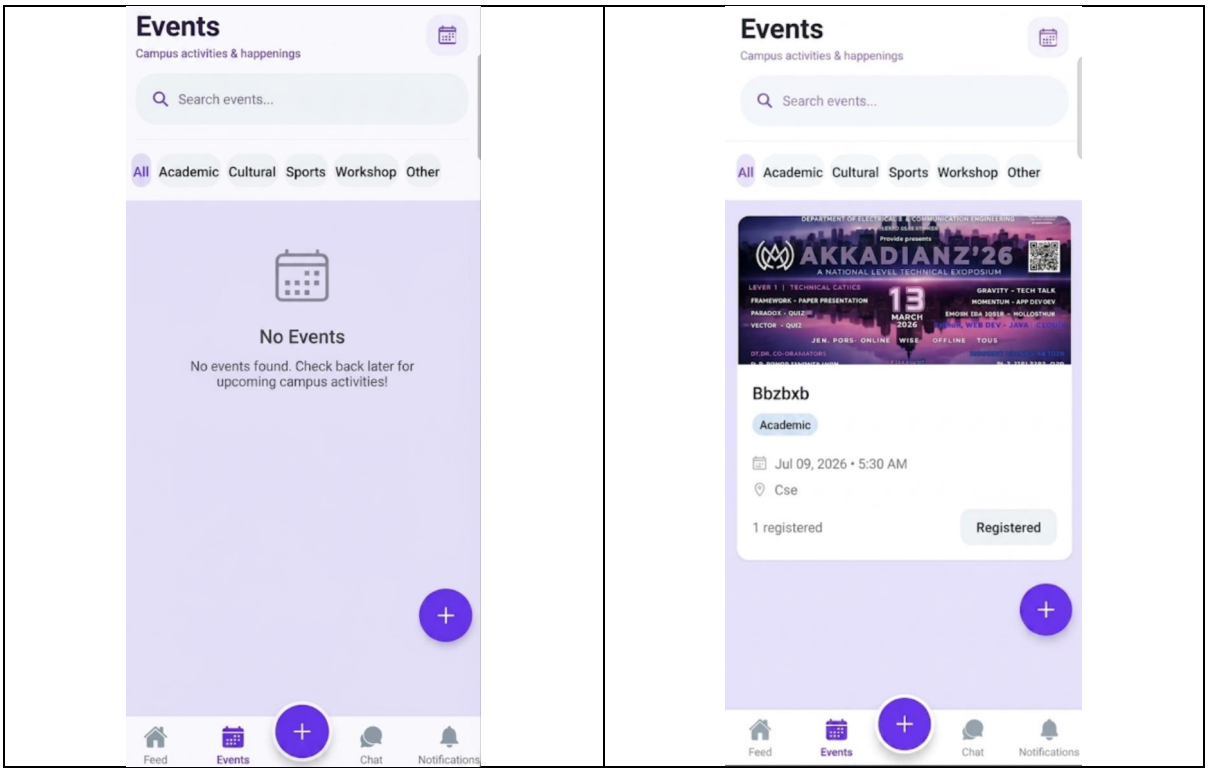

Figure 7a. Events screen — empty state with category filters, and a published event card with date, location, and registration status

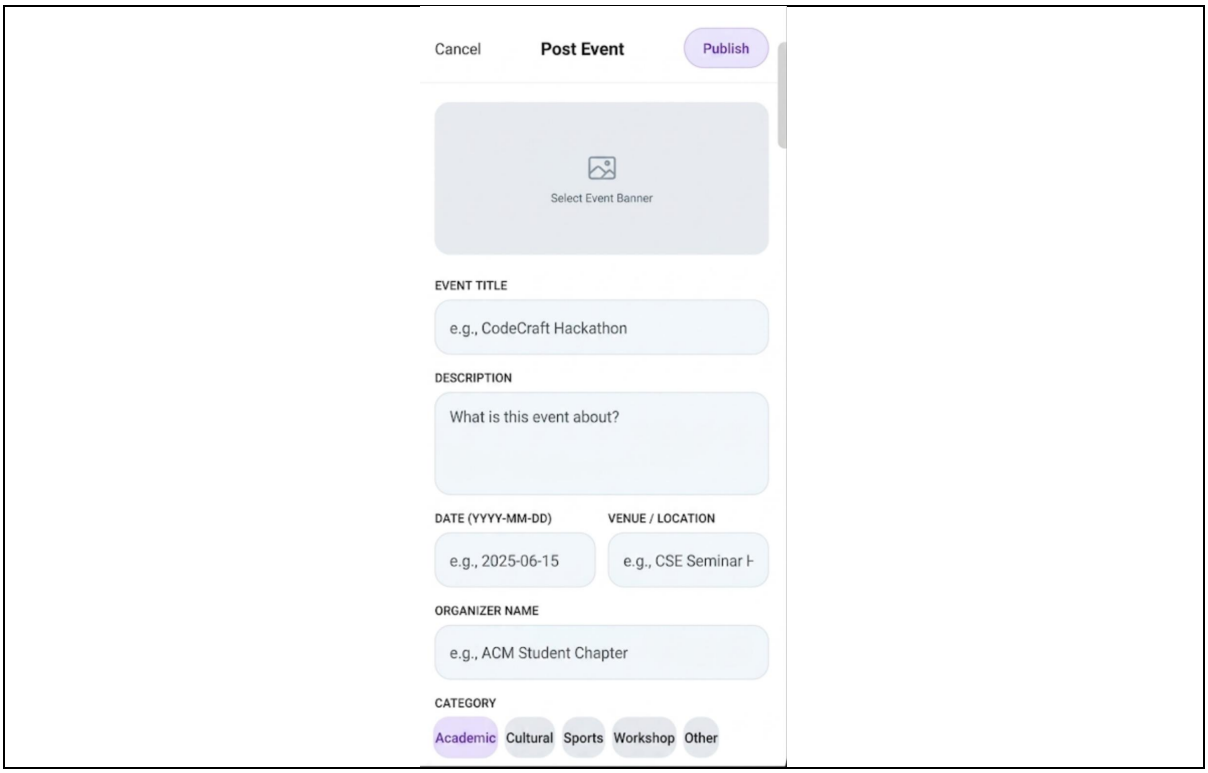

Figure 7b. Post Event form used by admin accounts to publish a new campus event

5.5. Real-Time Chat

Chat rooms are documents in the chatRooms collection with a type of group or direct and a members array. Each room's messages sub-collection stores individual messages with sender details, text or file content, a type field (text, image, or file), and a readBy array. Firestore's real-time listeners power live message delivery, and chatService handles sending messages and uploading shared files through Firebase Storage.

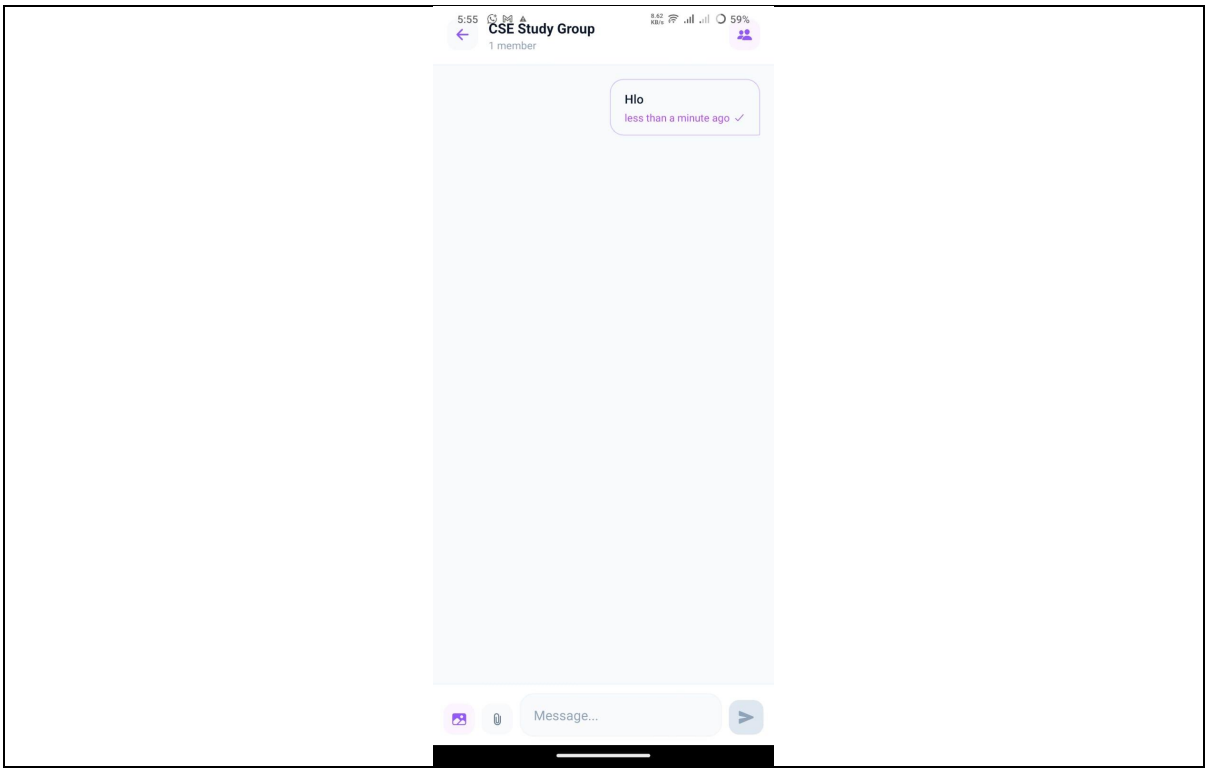

Figure 8. Real-time chat screen showing message bubbles and a shared file attachment

5.6. Notifications

Whenever a like, comment, event, announcement, or message occurs, notificationService writes a document to the notifications collection and dispatches a push notification through FCM and Expo Notifications, keyed to the user's stored fcmToken. For campus-wide announcements and targeted bulk alerts, the same registration tokens are mapped to topics managed by Amazon Pinpoint and Amazon SNS, allowing a single broadcast to reach the entire student body or a filtered segment. The notifications tab lists these events, marking them read via the isRead flag.

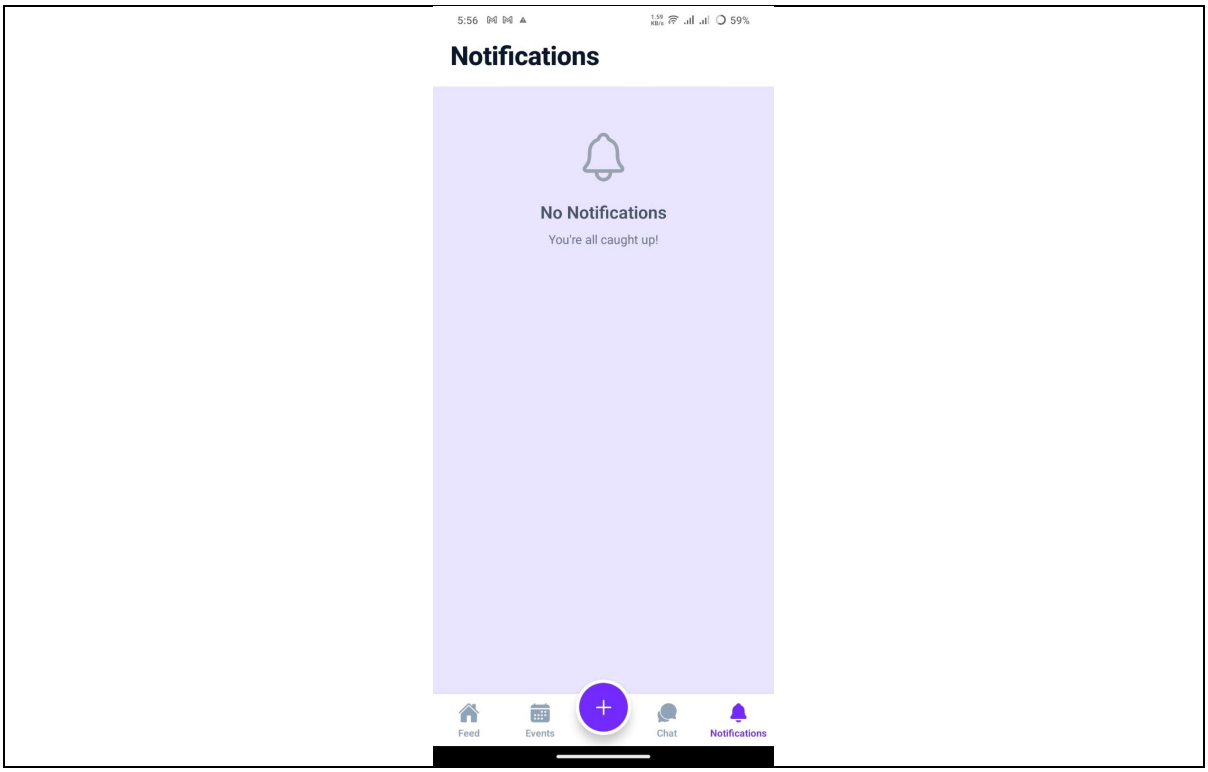

Figure 9. Notifications tab showing likes, comments, messages, and event alerts

5.7. Theming and Search

The useTheme hook and darkMode flag on the user document drive full dark-mode styling across all screens using NativeWind. A shared SearchBar component queries posts, events, and users, giving a single consistent search experience throughout the app.

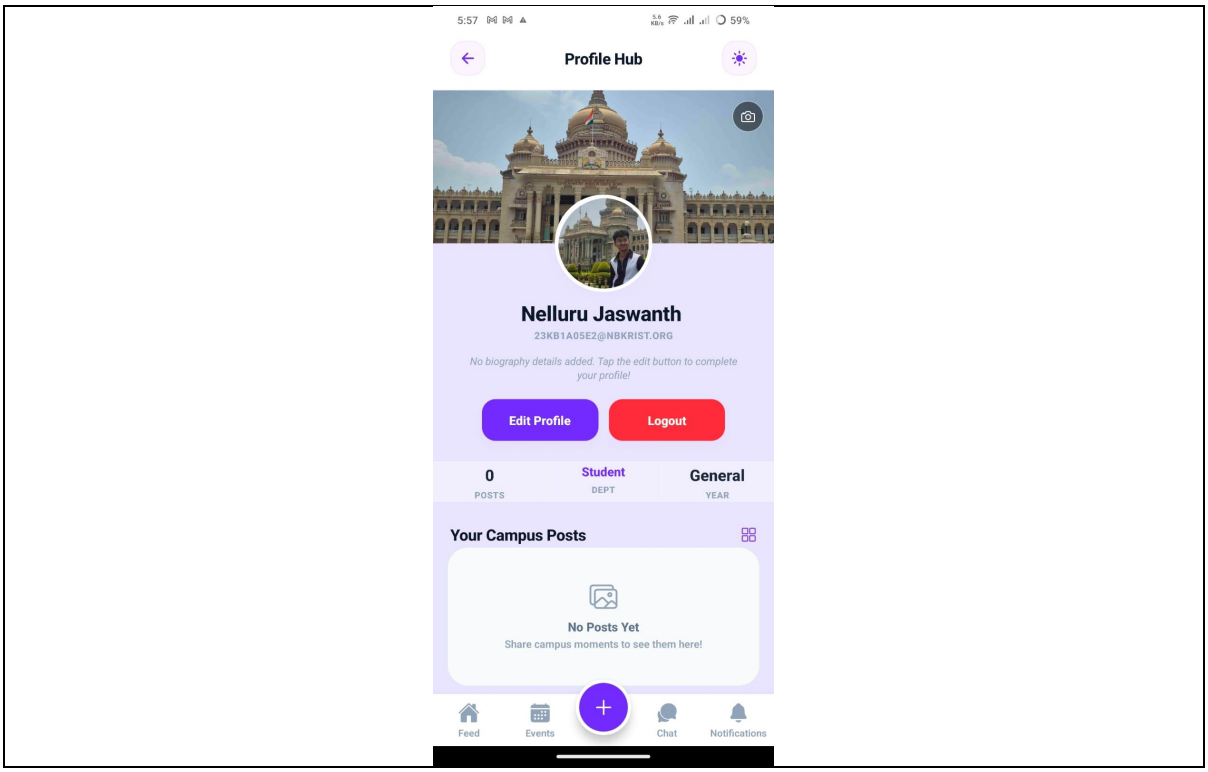

Figure 10. Profile screen (Profile Hub) with theme toggle, edit profile, and campus posts

5.8. AWS Moderation, Analytics & Notification Pipeline

The API Gateway + Lambda backend exposes REST endpoints consumed by the mobile client and by Firebase Cloud Functions for four purposes: moderating new posts, analyzing uploaded media, dispatching bulk notifications, and logging activity events. Each endpoint is backed by a dedicated Lambda function, keeping the moderation, analysis, and notification logic isolated and independently scalable.

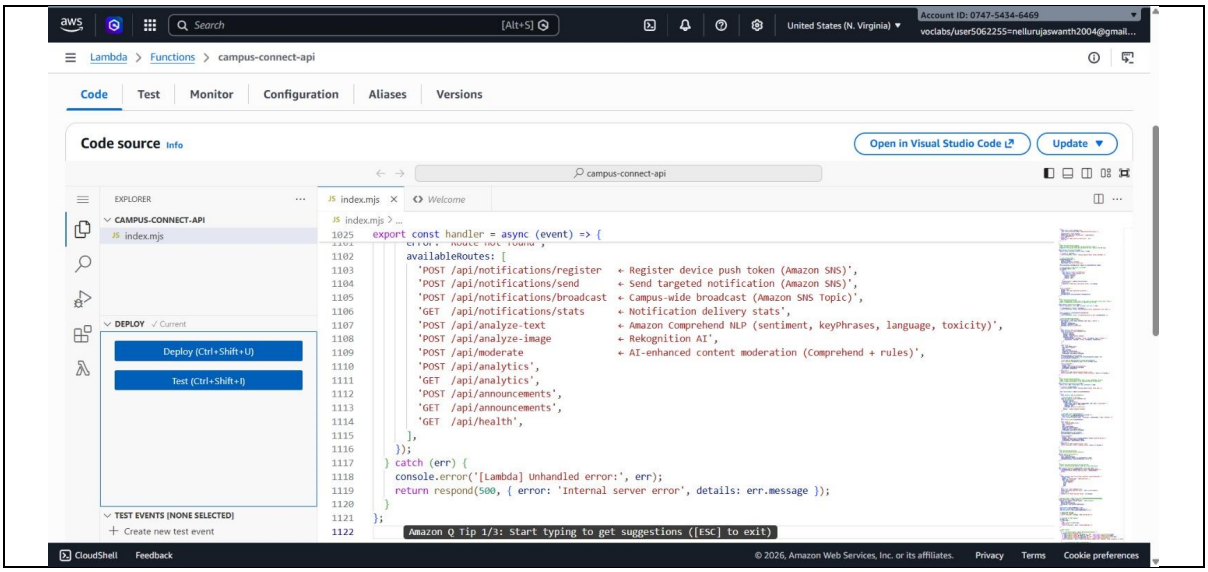

Figure 11a. Lambda function (campus-connect-api) source listing the notification, moderation, and analytics routes

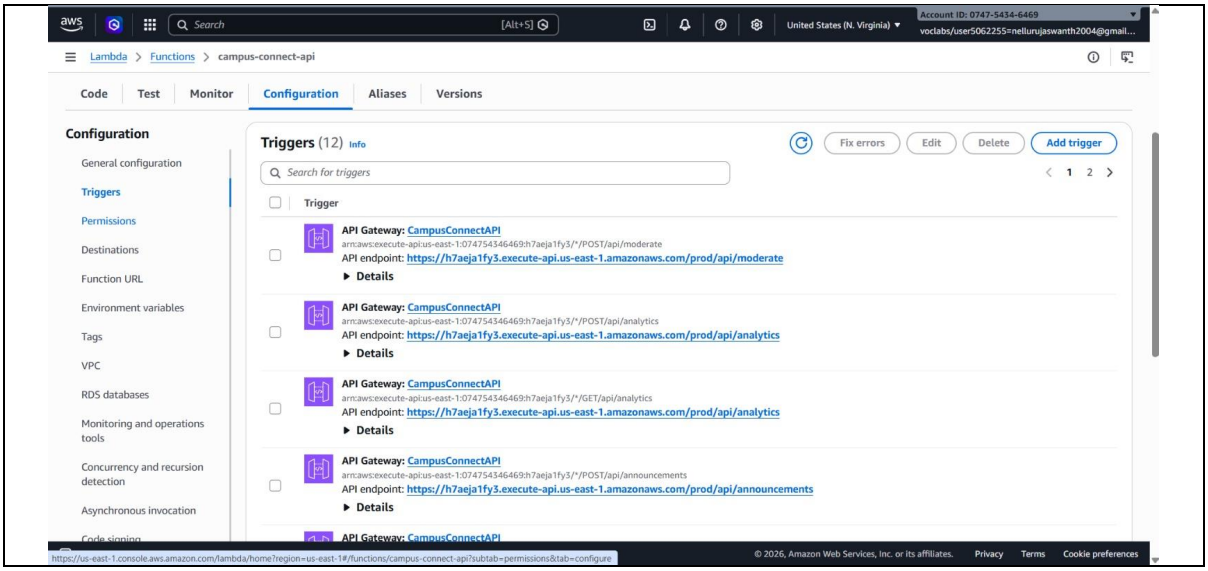

Figure 11b. API Gateway triggers configured on the campus-connect-api Lambda function

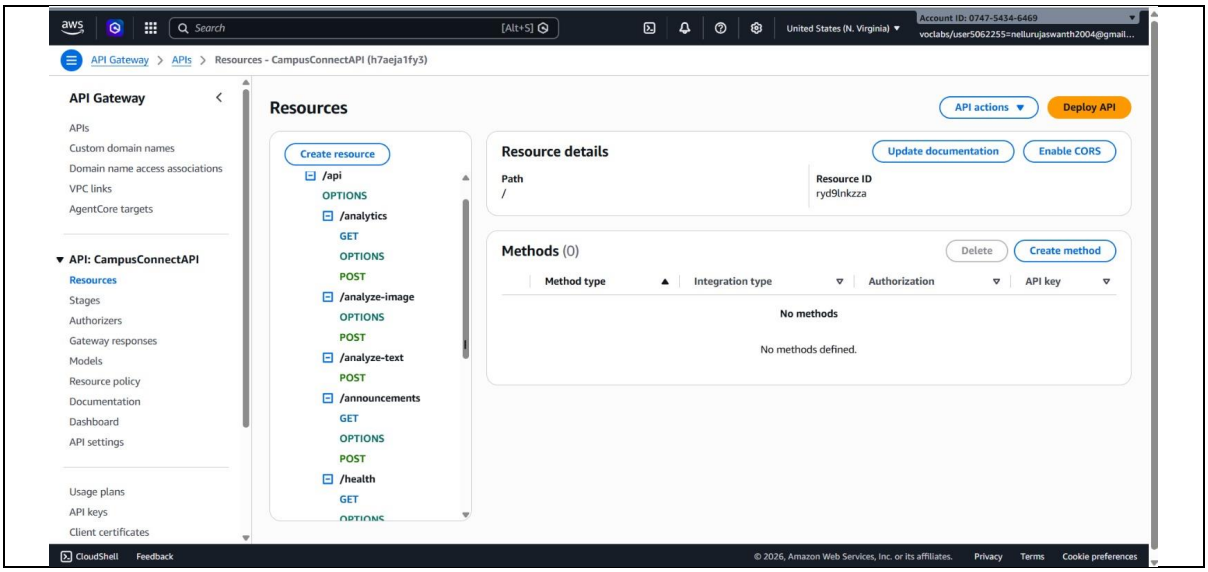

Figure 11c. API Gateway resource tree for CampusConnectAPI

Uploaded images and videos are passed to Amazon Rekognition, which returns moderation labels (flagging nudity or violence), OCR-extracted text, and descriptive tags used for search and categorization.

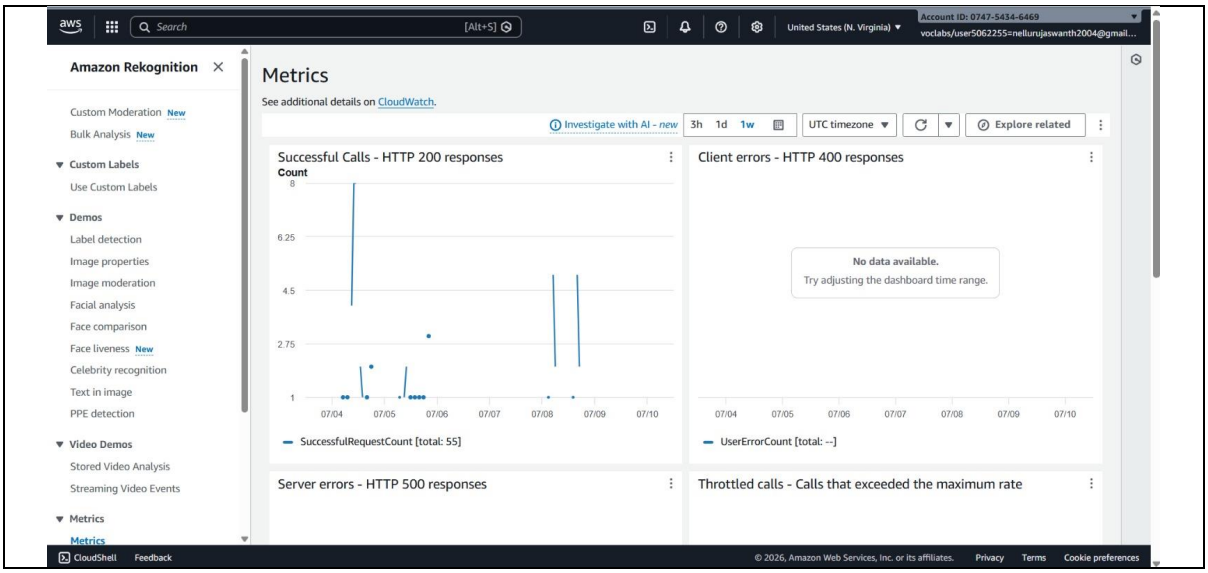

Figure 12. Amazon Rekognition console — API call metrics for moderation requests

Post text is passed to Amazon Comprehend, which returns a sentiment score, extracted key phrases, detected language, and a toxicity flag, allowing abusive or inappropriate posts to be withheld before they reach the feed.

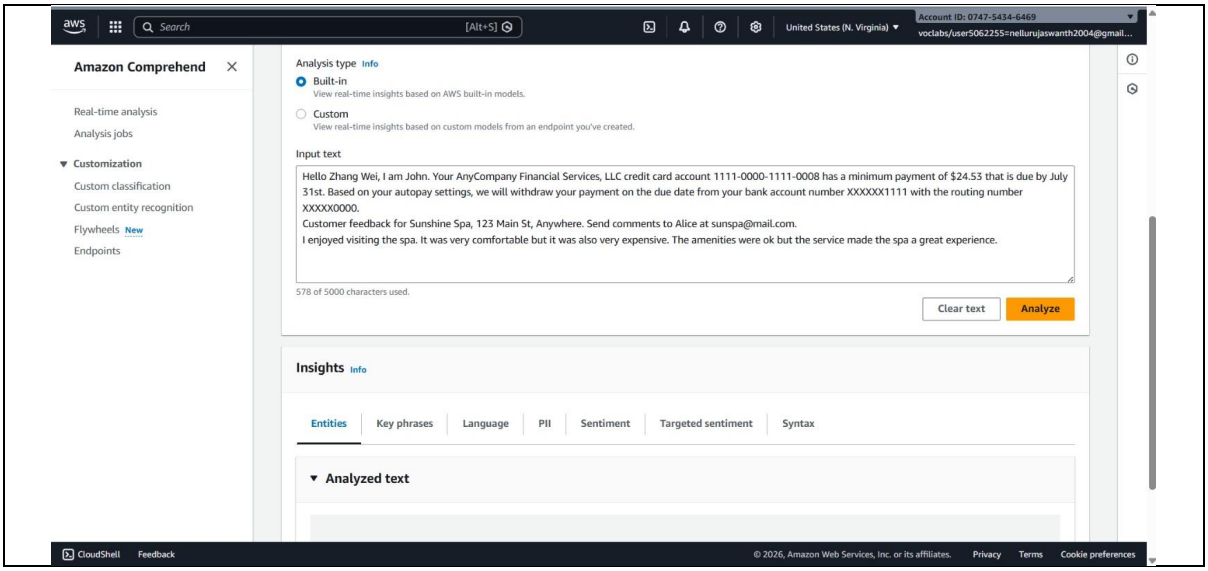

Figure 13. Amazon Comprehend console showing entity, key phrase, and sentiment analysis on sample text

Every view, like, post, and signup event is written as a JSON record to an Amazon S3 data lake bucket, partitioned by event type and date, ready to be queried with Amazon Athena and visualized in Amazon QuickSight dashboards.

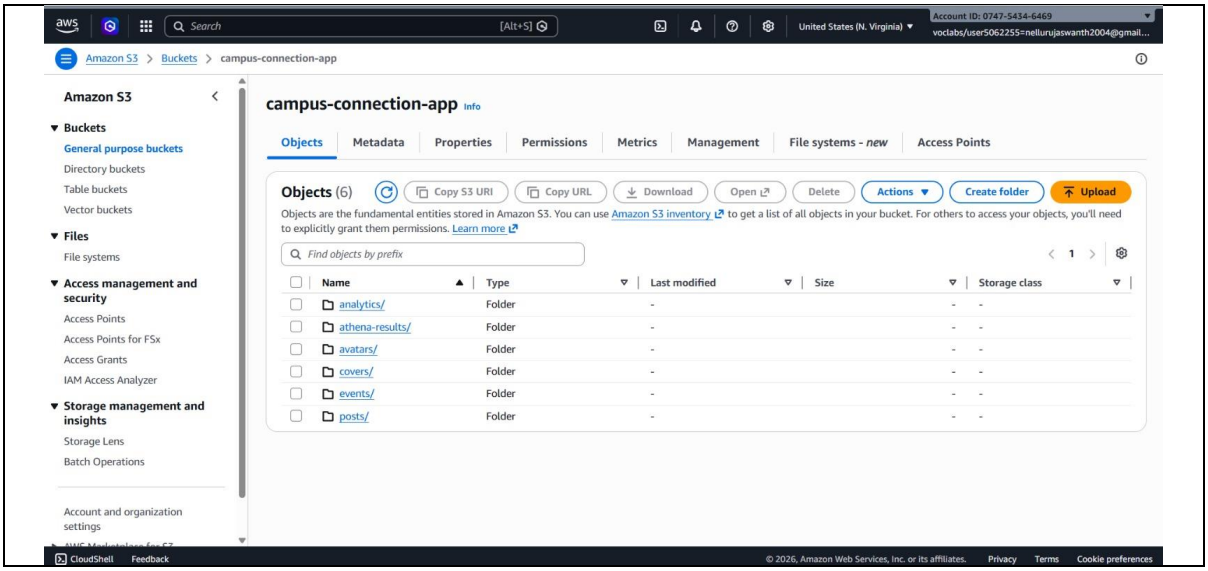

Figure 14. S3 bucket (campus-connection-app) showing analytics, athena-results, avatars, covers, events, and posts folders

Amazon Pinpoint and Amazon SNS manage the mapping between user device tokens and notification topics, allowing both individually targeted alerts and campus-wide broadcast announcements to be sent through a single API call.

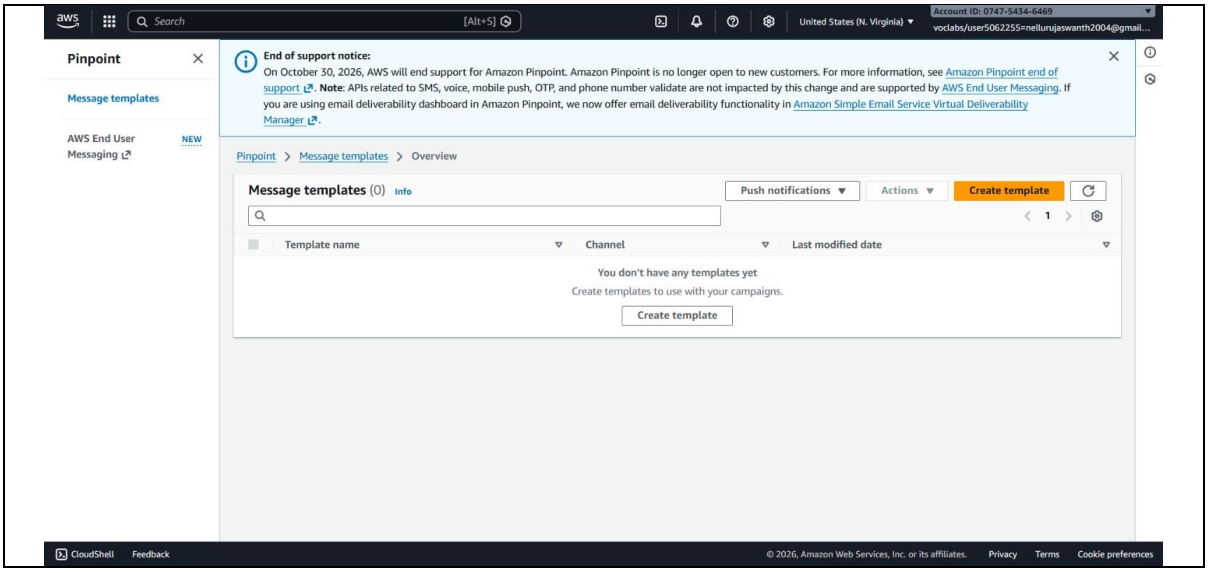

Figure 15. Amazon Pinpoint console — message templates for campus-wide push campaigns

5.9. Security and Deployment

Firestore Security Rules were authored and published to enforce ownership and role checks, and the app was configured for build and deployment through Expo Application Services (EAS), with Firebase configuration values supplied via environment variables so that development, testing, and production builds can point to separate Firebase projects if required. On the AWS side, IAM roles and policies restrict each Lambda function to the minimum permissions it needs — for example, the moderation function can invoke Rekognition and Comprehend but cannot write to the S3 analytics bucket, while the logging function can only append to S3 and has no access to Rekognition or Comprehend.

6. RESULTS

The implementation of Campus Connect achieved the outcomes defined at the start of the project. The application was tested across authentication, feed, events, chat, and notification flows to validate functionality, responsiveness, and reliability.

6.1. Real-Time Feed and Engagement

Posts created by any user appear instantly in other users' feeds through Firestore's real-time listeners. Likes and comments update live across all connected devices without requiring a manual refresh, confirming the responsiveness of the real-time data layer.

6.2. Event Discovery and Registration

Users can browse and filter events by category, and registration updates are reflected immediately in the event's registeredUsers list. Admin-only event creation was verified to be correctly restricted by both UI logic and Firestore Security Rules.

6.3. Real-Time Chat Performance

Both direct and group chats deliver messages in real time, with file sharing tested across image and document attachments. Read receipts, tracked through the readBy array, updated correctly as messages were viewed by recipients.

6.4. Push Notification Delivery

Push notifications for likes, comments, new messages, and event announcements were delivered promptly through Firebase Cloud Messaging and Expo Notifications, confirming reliable integration between Firestore triggers and the device notification layer.

6.5. Secure, Role-Based Access

Testing confirmed that Firestore Security Rules correctly limited each user to modifying only their own posts, profile, and chat messages, while administrative actions such as event creation were accessible only to accounts with isAdmin set to true.

6.6. Cross-Platform Consistency and Dark Mode

The application was verified to run consistently on both Android and iOS through Expo, with full dark mode theming applied uniformly across the feed, events, chat, notifications, and profile screens.

6.7. Content Moderation, Analytics & Bulk Notifications (AWS)

Test posts containing flagged imagery were correctly withheld from the feed by Amazon Rekognition's moderation labels, and posts containing abusive language were similarly blocked based on Amazon Comprehend's toxicity detection, confirming the moderation pipeline functions as intended before content reaches other users. Activity events logged to the Amazon S3 data lake were successfully queried through Athena, validating that usage patterns such as peak posting times and most-liked content categories can be derived without touching the production Firestore database. Campus-wide announcements sent through Amazon Pinpoint and SNS were delivered to all subscribed devices within the same latency range as individual Firebase notifications, confirming that the two notification paths can operate side by side without conflict.

6.8. User Experience

The system requires minimal friction for everyday use: users register once, and all subsequent posting, event registration, chatting, and notification handling happens seamlessly in real time. This confirms the application's readiness for real-world deployment within a campus community.

Overall, the results demonstrate that Campus Connect meets its intended goals of real-time communication, event coordination, secure access control, AI-assisted moderation, and a polished cross-platform user experience, with a stable foundation for further expansion.

6.9. Screenshots of the Output

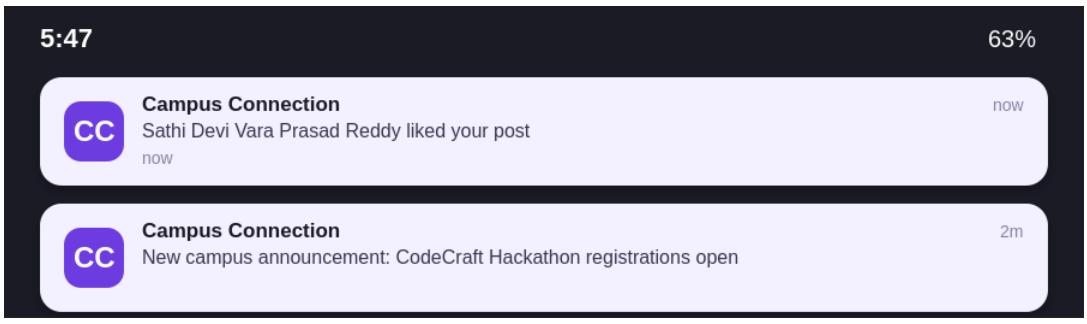

Figure 16. Illustrative mockup of push notifications for a like and a campus-wide announcement

Figure 17. Campus Connection app launch screen

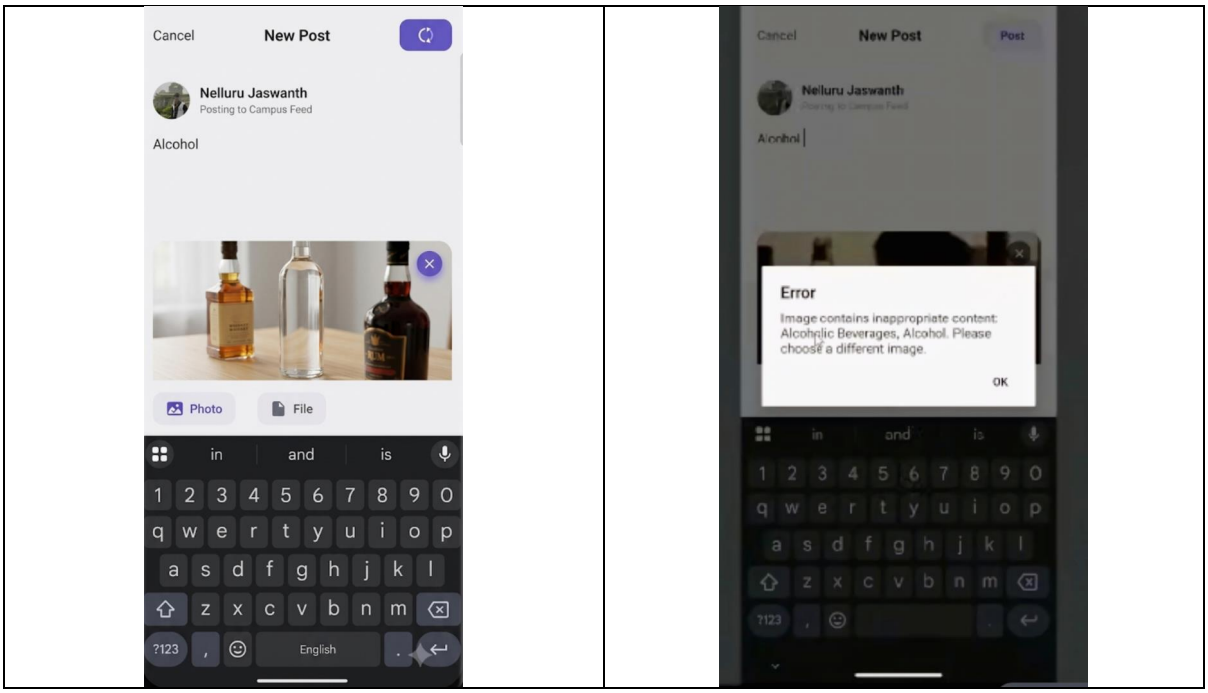

Figure 18. A post with a flagged image is blocked before publishing — Rekognition detects "Alcoholic Beverages, Alcohol" and the client rejects the post

7. CONCLUSION

The successful development of Campus Connect demonstrates the effectiveness of pairing a modern cross-platform mobile framework with a fully managed, serverless backend to solve a real, everyday problem for campus communities. By leveraging React Native with Expo alongside Firebase's Authentication, Firestore, Storage, and Cloud Messaging services, the application delivers a unified feed, event discovery and registration, real-time chat, and push notifications — all without provisioning or managing any backend infrastructure.

The project highlights the benefits of a backend-as-a-service architecture, including automatic scaling, real-time data synchronization, and reduced operational overhead. Zustand's lightweight state management, combined with a clean separation into stores, services, hooks, and components, kept the codebase modular and maintainable throughout development.

Careful attention was given to security and access control through Firestore Security Rules and role-based permissions, along with thorough testing of authentication, feed interactions, event registration, chat delivery, and notification flows across both Android and iOS devices.

Beyond its current functionality, the project lays a strong foundation for future enhancements, such as:

- An admin analytics dashboard for engagement and event insights
- Richer content moderation tools for posts and comments
- AI-assisted content and event recommendations based on user interests
- Deeper integration with institutional systems such as academic calendars and attendance

In conclusion, Campus Connect serves as a practical, scalable blueprint for building real-time, cloud-backed mobile applications for community platforms, showcasing how a serverless backend and a modern mobile client can be combined to deliver a secure, responsive, and engaging campus experience.